Introduction to Neural Networks

Convolutional Neural Networks

Juan Irving Vásquez
jivg.org

Centro de Innovación y Desarrollo Tecnológico en Cómputo
Instituto Politécnico Nacional

October 19, 2026©



Contents

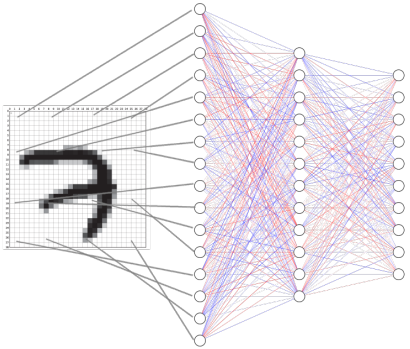
1. Motivation
2. Bidimensional convolution
3. Multichannel convolution
4. Stride, padding and stacking

Roadmap

1. Motivation
2. Bidimensional convolution
3. Multichannel convolution
4. Stride, padding and stacking

The fully connected networks

The FCN can be adapted to recognize patterns in several dimensions.



"The response of most of these models, however, was severely affected by the shift in position and/or by the distortion in shape of the input patterns"[1]

[1] Fukushima, Kunihiko. "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position." Biological cybernetics 36.4 (1980): 193-202.

Convolutional Neural Networks (CNN)

- Position invariance of features



Convolutional Neural Networks (CNN)

- Position invariance of features



- CNNs share their parameters across the space

Template matching

- Template

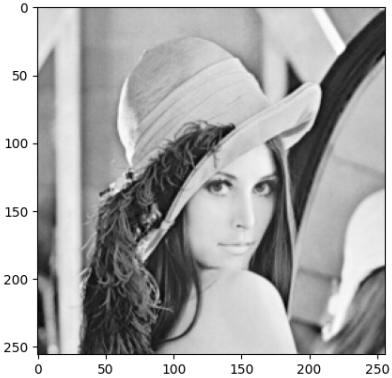


- Picture



Convolution property

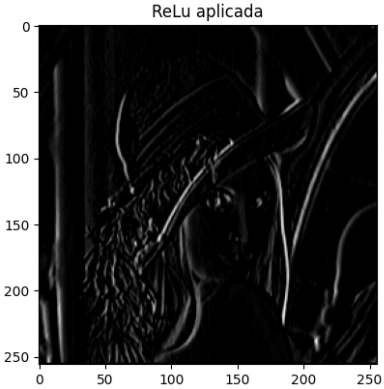
Graphical motivation



(a) Input image (I)

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

(b) Kernel



(c) Output (G)

Convolutional Neural Networks insight

The convolutional neural networks were introduced by LuCun in 1989.

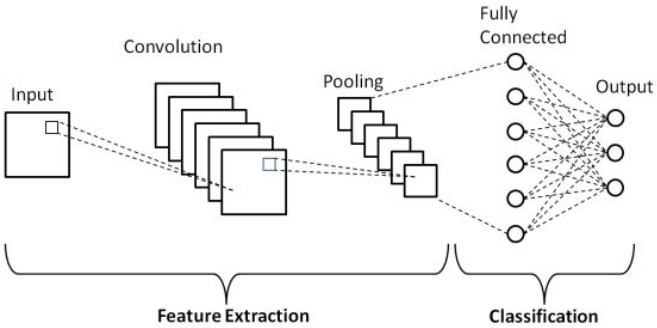


Figure: Scheme of a convolutional neural network for classification.

Roadmap

1. Motivation
2. Bidimensional convolution
3. Multichannel convolution
4. Stride, padding and stacking

Convolution Inputs

- Input

$$I = \begin{bmatrix} x_{1,1} & \dots & x_{1,m} \\ \vdots & \ddots & \vdots \\ x_{m,1} & \dots & x_{l,m} \end{bmatrix} \tag{1}$$

  where $I \in \mathbb{R}^{m \times m}$, y

- Kernel

$$\mathbf{K} = \begin{bmatrix} w_{1,1} & \dots & w_{1,k} \\ \vdots & \ddots & \vdots \\ w_{k,1} & \dots & w_{k,k} \end{bmatrix} \tag{2}$$

  where $\mathbf{K} \in \mathbb{R}^{k \times k}$, $k = (2l + 1)$ y $k < m$.

- Given two functions, I and $\mathbf{K}$, the convolution produces a new function that changes the shape of the first one according the to second one.

$$G[i,j] = \sum_{u=-l}^{l} \sum_{v=-l}^{l} \mathbf{K}[u,v] I[i-u, j-v] \tag{3}$$

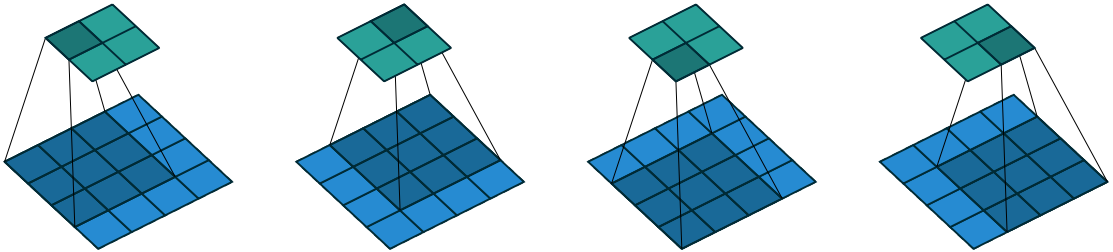
- It is very useful to find patterns.

- Given two functions, I and $\mathbf{K}$

$$G[i,j] = \sum_{u=-l}^{l} \sum_{v=-l}^{l} \mathbf{K}[u,v] I[i+u, j+v] \tag{4}$$

- The cross correlation is more used in machine learning.

Convolution layer



(a) Step 1 (b) Step 2 (c) Step 3 (d) Step 4

Figure: Convolution: a kernel is moved inside the input to create a new matrix.

https://github.com/vdumoulin/conv_arithmetic/raw/master/gif/no_padding_no_strides.gif

Convolutional layer

Composed by:

- Convolution [2]

$$\texttt{Conv2D}(K, I)[i, j] = b + \sum_{u=-l}^{l} \sum_{v=-l}^{l} \mathbf{K}[u, v] I[i + u, j + v] \tag{5}$$

- Activation

$$\hat{Y}[i, j] = f(\texttt{Conv2d}(\mathbf{K}, I)[i, j]) \tag{6}$$

[2] Similar to Pytorch Conv2D

Convolution 2D

Illustration

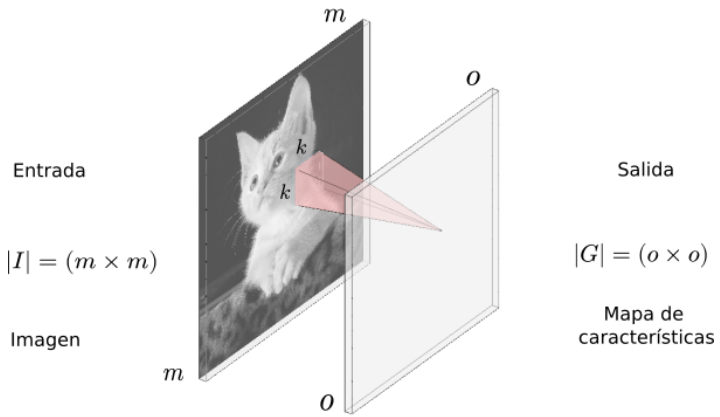


Figure: Sizes of a 2D convolution

Characterization

- Output size

$$o = 1 + (m - \mathrm{k}) \tag{7}$$

- Amount of parameters

$$|\theta| = \mathrm{k}_1 \cdot \mathrm{k}_2 + 1 \tag{8}$$

Roadmap

1. Motivation
2. Bidimensional convolution
3. Multichannel convolution
4. Stride, padding and stacking

Multichannel input

- In the majority of cases the input to a convolution layer has more than one channel, e.g. RGB images.

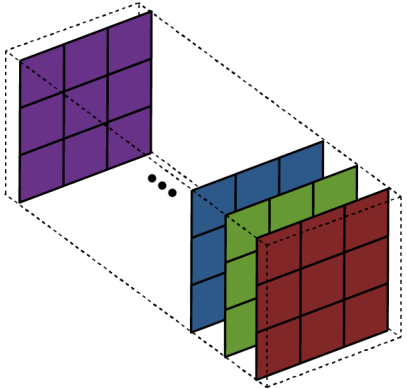


Figure: Illustration of an input image with size $(c \times m \times m)$. c indicates the number of channels.

Multichannel convolution

- The kernel size is $(c \times k \times k)$
- The kernel's width and height are usually smaller than the input but the depth is the same than the input number of channels.
- Computation:

$$\texttt{Conv2DMulticanal}(K, I, b)[i, j] = b + \sum_{w=1}^{c} \sum_{u=-l}^{l} \sum_{v=-l}^{l} K[w, u, v] I[w, i + u, j + v] \quad (9)$$

Multichannel convolution

Illustration

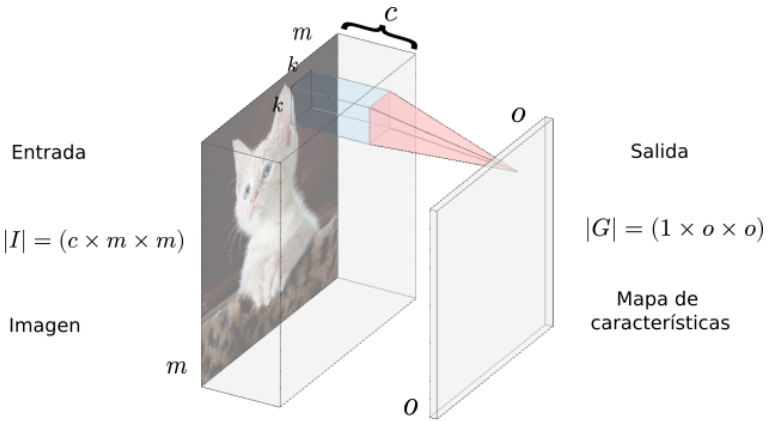
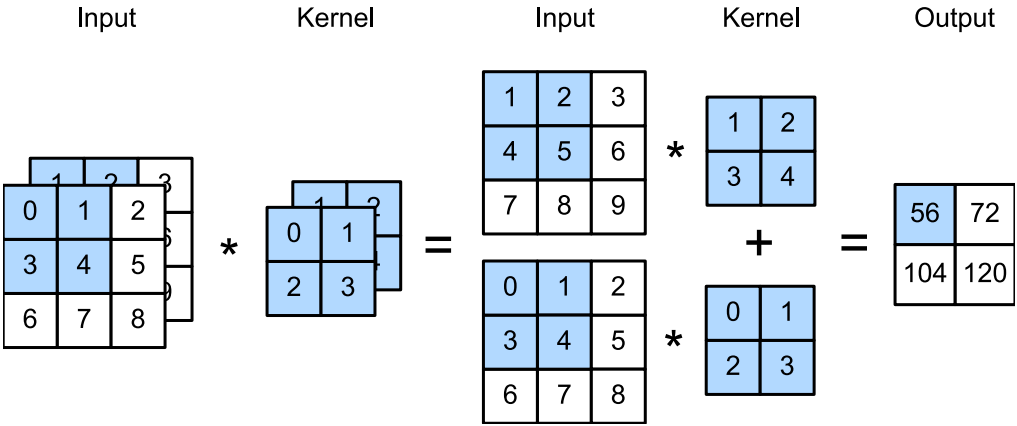


Figure: Convolution sizes of the multichannel convolution. The kernel's width and height are usually smaller than the input but the depth is the same than the input number of channels

Multichannel convolution example

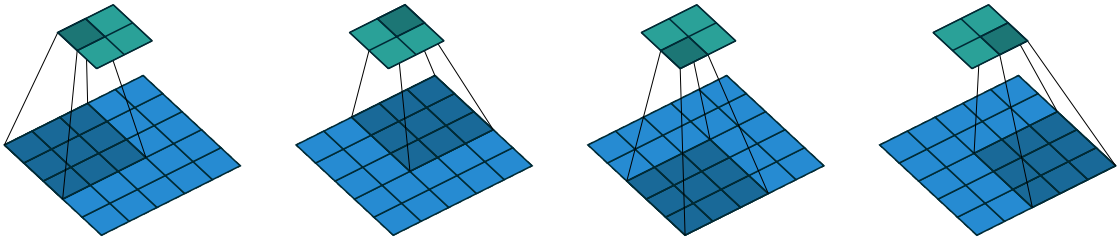


Figure

Roadmap

1. Motivation
2. Bidimensional convolution
3. Multichannel convolution
4. Stride, padding and stacking

CNN Stride



(a) Subfig. 1 (b) Subfig. 2 (c) Subfig. 3 (d) Subfig. 4

Figure: Visualización del proceso de convolución con salto $s = 2$. El salto o *stride* provoca un desplazamiento mayor del kernel.

CNN Stride (2)

output size

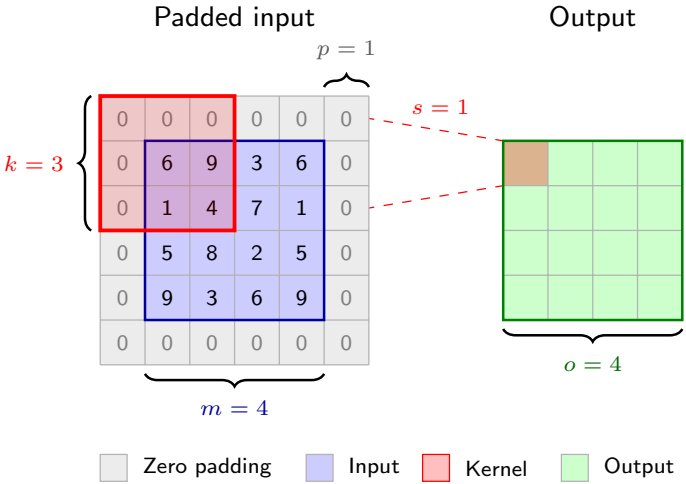
- The stride is defined by s
- The output feature width size is then:

$$o = \lfloor \frac{m - k}{s} \rfloor + 1 \tag{10}$$

where m is the input, k the kernel size, o is the output size,

- The same applies for height

CNN Padding


Padded input
Output
$p = 1$
$k = 3$
$s = 1$
$m = 4$
$o = 4$
Zero padding
Input
Kernel
Output

For any input (m), kernel size (k), padding (p) and stride (s):

$$o = \lfloor \frac{m + 2p - k}{s} \rfloor + 1 \tag{11}$$

Convolution layer with several kernels

- A convolutional layer can apply several (d) kernels to the same input.
- The output of each convolution is stack in the feature map.

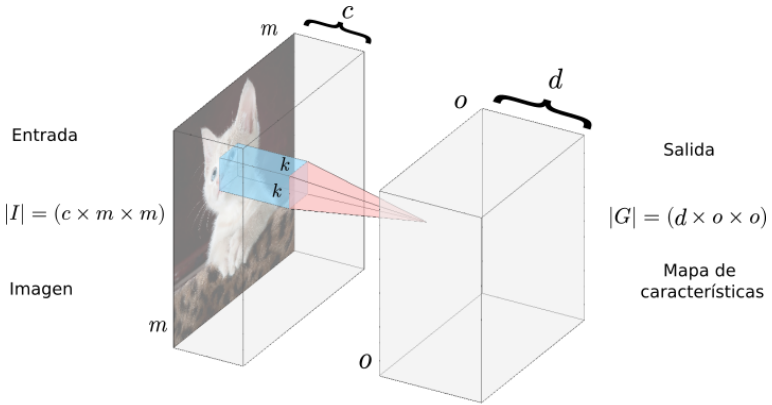


Figure: The depth of the feature map is the same as the number of applied kernels.

Exercise

- Given: An input of $(3 \times 28 \times 28)$, 8 kernels of $(3 \times 3 \times 3)$, calculate the size of the feature maps:

Padding	p	Stride (s)	Depth	Width	Height
valid	0	1			
same		1		28	28
valid	0	2			

Max Pooling

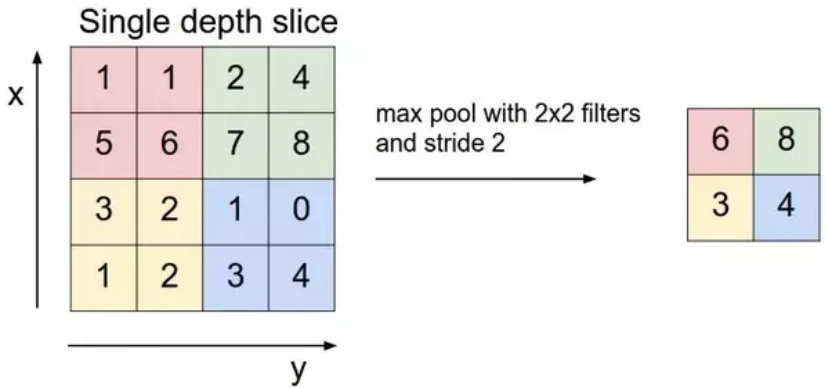


Figure: Pooling reduces the size of the feature map.

Pooling

- Given input (i), window (v) and stride (s), the output size is

$$o = \lfloor \frac{i - v}{s} \rfloor + 1 \tag{12}$$

- Some characteristics
  - Weights free
  - A more accurate model
  - Forward pass more expensive
  - More hyper-parameters

1×1 convolution

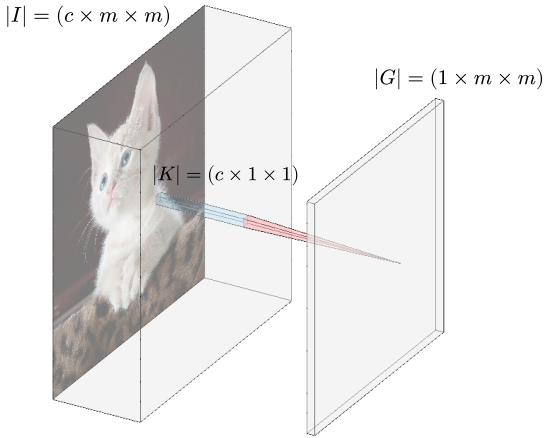


Figure: A 1×1 convolution adds more parameters without modifying the structure.

Inception module (optional)

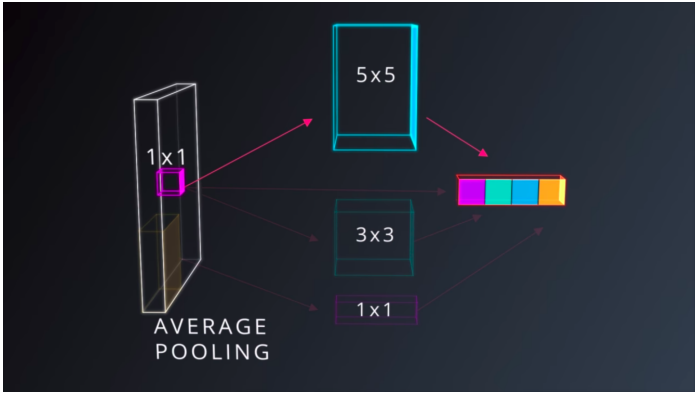


Figure: Why to choose between different operations? Let make all!.

CNN Classification

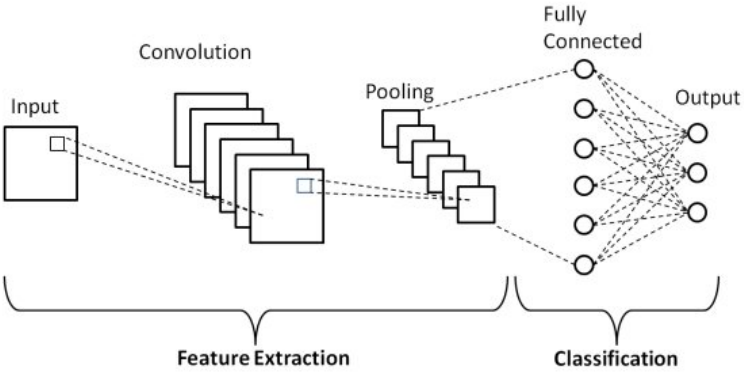
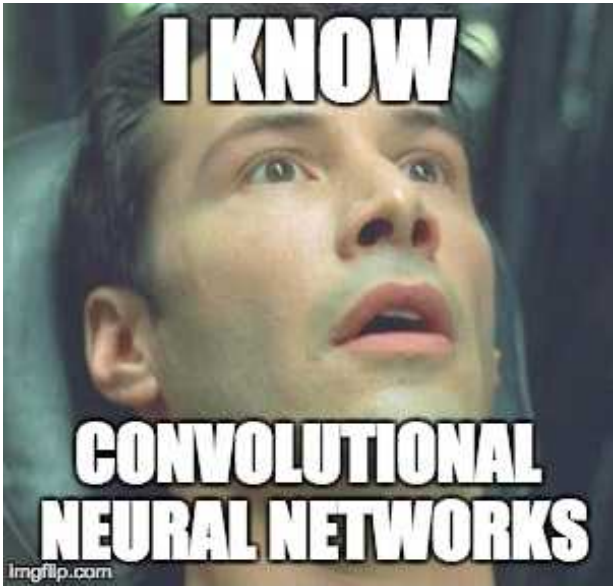


Figure: In a CNN several convolutions are applied before a classification stage.


I KNOW
CONVOLUTIONAL
NEURAL NETWORKS
imgflip.com

References

- Dumoulin, Vincent, and Francesco Visin. "A guide to convolution arithmetic for deep learning." arXiv preprint arXiv:1603.07285 (2016).
- Juan Irving Vasquez, Introducción a las redes neuronales, Instituto Politécnico Nacional.
- Skansi, S. (2018). Introduction to Deep Learning: from logical calculus to artificial intelligence. Springer.